

228. Summary Ranges

<https://leetcode.com/problems/summary-ranges/description/>

Дата: 02.02.2026

Автор: Андрей Трушин, Глеб Михайлов

Для курса "Алгоритмы с Глебом Михайловым" 🐯

Где дают: Яндекс, VK

1. Условие задачи

Нам дают отсортированный массив уникальных целых чисел `nums`. Нужно сгруппировать идущие подряд числа в диапазоны.

Например, если в массиве есть 0, 1, 2, мы превращаем их в строку "0->2". Если число стоит особняком, например 7, то просто записываем его как "7". На выходе должен получиться список таких строк.

Важные детали:

- Массив уже отсортирован по возрастанию.
- Все числа уникальны.
- Длина массива от 0 до 20 (в этой версии задачи).

2. Решение

Идея решения:

Тут всё просто: мы идем по массиву слева направо. Раз массив отсортирован и числа уникальны, то у "соседей" в одном диапазоне разница должна быть ровно 1.

Мы фиксируем начало потенциального диапазона и бежим вперед, пока следующее число больше текущего ровно на единицу. Как только эта цепочка рвется — фиксируем конец диапазона, записываем результат и начинаем новый поиск с текущего места.

Алгоритм по шагам:

1. Создаем пустой список для ответов.
2. Запускаем цикл `i` от 0 до конца массива.
3. Внутри цикла запоминаем текущее число как начало диапазона (`start = nums[i]`).
4. Запускаем вложенный цикл (или просто двигаем `i`), пока `nums[i + 1]` равен `nums[i] + 1`.
Важно не вылететь за границы массива.
5. Когда цикл остановился, проверяем:
 - Если `start` совпадает с `nums[i]`, значит, в диапазоне всего одно число. Пишем

просто "число".

- Если не совпадает, пишем "start->nums[i]".

6. Переходим к следующему элементу.

Визуализация:

Возьмем пример nums = [0, 1, 2, 4, 5, 7]

Шаг (i)	nums[i]	start	Условие (nums[i]+1 == nums[i+1])	Конец диапазона ?	Результат
0	0	0	Да (1 == 0+1)	Нет	
1	1	0	Да (2 == 1+1)	Нет	
2	2	0	Нет (4 != 2+1)	Да	["0->2"]
3	4	4	Да (5 == 4+1)	Нет	
4	5	4	Нет (7 != 5+1)	Да	["0->2", "4->5"]
5	7	7	Нет (конец массива)	Да	["0->2", "4->5", "7"]

Ответ: ["0->2", "4->5", "7"]

Код на Python:

```
class Solution:
    def summaryRanges(self, nums: list[int]) -> list[str]:
        res = []
        i = 0
        n = len(nums)

        while i < n:
            start = nums[i]

            # Двигаем указатель, пока числа идут подряд (+1)
            while i + 1 < n and nums[i + 1] == nums[i] + 1:
                i += 1

            # Если i не сдвинулся, значит в диапазоне одно число
```

```

if start == nums[i]:
    res.append(str(start))
else:
    res.append(f"{start}->{nums[i]}")

# Переходим к следующему числу после найденного диапазона
i += 1

return res

```

Краевые случаи:

- **Пустой массив ([])**: Цикл `while i < n` просто не запустится, вернем пустой список. Это корректно.
- **Один элемент ([5])**: Пройдет один раз, `start` будет равен `nums[i]`, добавит "5".
- **Числа с большим разрывом ([1, 10, 100])**: Каждый раз цепочка будет рваться сразу, получим три отдельных строки.

3. Сложность алгоритма

Пошаговый анализ:

Мы проходим по массиву ровно один раз. Несмотря на то, что у нас циклы вложены (`while` внутри `while`), внешний указатель `i` только увеличивается. Мы никогда не возвращаемся назад. Каждое число из `nums` мы "трогаем" максимум пару раз.

Итог:

- **По времени:** $O(N)$, где N — количество чисел в массиве. Линейный проход.
- **По памяти:** $O(1)$, если не считать результирующий список строк (обычно на собеседах память под ответ не учитывают в доп. памяти). Если считать ответ, то $O(N)$ в худшем случае, когда каждое число — отдельный диапазон.

4. Финал

Задача учит аккуратно работать с индексами и понимать, что два вложенных цикла — это не всегда $O(N^2)$. Главное — смотреть на то, как ведут себя указатели.

